topic_02_thread_safe_bounded_queue

Theory, Edge Cases & Code Examples

- ✓ The Producer-Consumer Problem
- ✓ Synchronization Primitives
- ✓ **std::mutex**
- ✓ **std::condition_variable**
- ✓ Design Considerations

→ Swipe to learn →

topic_02_thread_safe_bounded_queue

Theory

📌 1. The Producer-Consumer Problem

The **bounded buffer** (or **producer-consumer**) problem is a classic synchronization challenge:

- **Producers** add items to a queue
- **Consumers** remove items from the queue
- The queue has a **maximum capacity** (bounded)
- Multiple threads may produce/consume simultaneously

Challenges:
- **Race conditions**: Multiple threads accessing shared data
- **Buffer full**: Producers must wait when queue is full
- **Buffer empty**: Consumers must wait when queue is empty
- **Deadlock**: Improper locking can cause threads to wait forever
- **Spurious wakeups**: Condition variables may wake without actual condition change

---

topic_02_thread_safe_bounded_queue

Theory

📌 2. Synchronization Primitives

topic_02_thread_safe_bounded_queue

Theory

📌 **std::mutex**

Provides mutual exclusion - ensures only one thread accesses critical section at a time.

```
std::mutex mtx;
mtx.lock();    // Acquire lock
// Critical section
mtx.unlock();  // Release lock

// RAII approach (preferred):
std::lock_guard<std::mutex> lock(mtx);  // Auto-unlocks on scope exit
```

topic_02_thread_safe_bounded_queue

Theory

📌 **std::condition_variable**

Allows threads to wait for a condition to become true:

```
std::condition_variable cv;
std::mutex mtx;
bool ready = false;

// Waiting thread:
std::unique_lock<std::mutex> lock(mtx);
cv.wait(lock, []{ return ready; });  // Atomically unlocks and waits

// Notifying thread:
{
    std::lock_guard<std::mutex> lock(mtx);
    ready = true;
}
cv.notify_one();  // or cv.notify_all()
```

Key Points:

- `wait()` atomically releases mutex and blocks
- When woken, re-acquires mutex before returning
- Always use predicate version to handle spurious wakeups
- Requires `std::unique_lock` (not `lock_guard`) for unlocking

📌 3. Design Considerations

topic_02_thread_safe_bounded_queue

Theory

📌 **Bounded vs Unbounded**

- **Bounded**: Fixed capacity, producers block when full (prevents memory exhaustion)
- **Unbounded**: Grows dynamically, may cause OOM

topic_02_thread_safe_bounded_queue

Theory

📌 **FIFO Semantics**

- First-In-First-Out ordering guaranteed
- Use `std::queue` or circular buffer internally

topic_02_thread_safe_bounded_queue

Theory

📌 **Exception Safety**

- Operations should provide strong exception guarantee
- Lock must be released even if exceptions occur (use RAII)

topic_02_thread_safe_bounded_queue

Theory

📌 **Timeout Support**

- `wait_for()` / `wait_until()` for timed waits
- Prevents indefinite blocking

---

topic_02_thread_safe_bounded_queue

Theory

📌 4. Implementation Strategy

Core Components:

1. **Internal container** (std::queue or circular buffer)
2. **Mutex** for protecting shared state
3. **Two condition variables**: one for "not full", one for "not empty"
4. **Capacity tracking**: current size vs max size

Operations:

- `push()`: Wait if full, add item, notify consumers
- `pop()`: Wait if empty, remove item, notify producers
- `try_push()` / `try_pop()`: Non-blocking variants
- `size()`, `empty()`, `full()`: Thread-safe queries

---

topic_02_thread_safe_bounded_queue

Edge Cases

📌 1. Spurious Wakeups

Problem: Condition variables may wake without actual notification.

```
// WRONG - vulnerable to spurious wakeups:
not_empty_.wait(lock);
T value = queue_.front();  // May be empty!

// CORRECT - always use predicate:
not_empty_.wait(lock, [this]() { return !queue_.empty(); });
T value = queue_.front();  // Safe
```

Why it happens: OS-level optimizations, signal handling.

---

topic_02_thread_safe_bounded_queue

Edge Cases

📌 2. Deadlock with Multiple Locks

Problem: Acquiring locks in different orders causes deadlock.

```
// Thread 1:
lock(mutex_A);
lock(mutex_B);

// Thread 2:
lock(mutex_B);  // ← Deadlock!
lock(mutex_A);
```

Solution:

- Always acquire locks in same order
- Use `std::scoped_lock` for multiple mutexes (C++17)
- Or design with single mutex

---

topic_02_thread_safe_bounded_queue

Edge Cases

📌 3. Exception During Push/Pop

Problem: If copy constructor throws, state may be inconsistent.

```
T pop() {
    std::unique_lock<std::mutex> lock(mutex_);
    not_empty_.wait(lock, [this]() { return !queue_.empty(); });

    T value = queue_.front();  // Copy may throw
    queue_.pop();              // Never reached!
    return value;
}
```

Solution: Use move semantics or strong exception guarantee:

```
T value = std::move(queue_.front());  // Move (typically noexcept)
queue_.pop();  // Only pop after successful move
```

---

topic_02_thread_safe_bounded_queue

Edge Cases

📌 4. Busy-Waiting (Performance)

Problem: Spinning on `try_pop()` wastes CPU.

```
// BAD - busy-wait loop:
while (true) {
    auto item = queue.try_pop();
    if (item) process(*item);
    // CPU at 100%!
}

// GOOD - use blocking pop:
while (true) {
    T item = queue.pop();  // Sleeps when empty
    process(item);
}
```

topic_02_thread_safe_bounded_queue

Edge Cases

📌 5. Forgetting to Notify

Problem: Missing notification causes waiting threads to hang forever.

```
void push(const T& value) {
    std::lock_guard<std::mutex> lock(mutex_);
    queue_.push(value);
    // Forgot not_empty_.notify_one(); ← Consumer hangs!
}
```

Best Practice: Always pair state changes with notifications.

---

topic_02_thread_safe_bounded_queue

Code Examples

📌 1. Basic Producer-Consumer (1/2)

This example demonstrates the **classic producer-consumer pattern** with multiple producers and multiple consumers working concurrently on a shared bounded queue. This is a fundamental concurrency pattern used extensively in multi-threaded applications.

What this code does:
- Creates 2 producer threads that each push 10 items to the queue
- Creates 2 consumer threads that each pop 10 items from the queue
- Producers write at 100ms intervals, consumers read at 150ms intervals
- The queue has capacity 5, so producers will block when full (consumers are slower)
- All threads safely share the same queue without data races

Key concurrency concepts demonstrated:
- **Thread synchronization**: Multiple threads coordinate access to shared queue
- **Blocking behavior**: When queue is full (5 items), producers wait; when empty, consumers wait
- **FIFO ordering**: Items are consumed in the order they were produced
- **Interleaved execution**: Output shows threads executing in non-deterministic order (this is normal!)
- **Pass by reference**: `std::ref(queue)` ensures all threads share the same queue object

Why the timing matters:
- Producers push every 100ms → 20 items/second total (2 threads × 10 items each)
- Consumers pop every 150ms → 13.3 items/second total (2 threads)
- **Producers are faster** → queue fills up → producers block → demonstrates bounded buffer back-pressure

Real-world application:
- **Web server**: Producer threads accept connections, consumer threads process requests
- **Data pipeline**: Producers read from disk/network, consumers process and write results
- **Task scheduler**: Producers generate tasks, worker threads consume and execute them

Output (interleaved):

```
#include <thread>
#include <iostream>

void producer(BoundedQueue<int>& queue, int id) {
// ...continues on next slide
```

topic_02_thread_safe_bounded_queue

Code Examples

📌 1. Basic Producer-Consumer (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    for (int i = 0; i < 10; ++i) {
        int value = id * 100 + i;
        queue.push(value);
        std::cout << "Producer " << id << " pushed: " << value << '\n';
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
}

void consumer(BoundedQueue<int>& queue, int id) {
    for (int i = 0; i < 10; ++i) {
        int value = queue.pop();
        std::cout << "Consumer " << id << " popped: " << value << '\n';
        std::this_thread::sleep_for(std::chrono::milliseconds(150));
    }
}

int main() {
    BoundedQueue<int> queue(5);  // Capacity: 5

    std::thread prod1(producer, std::ref(queue), 1);
    std::thread prod2(producer, std::ref(queue), 2);
    std::thread cons1(consumer, std::ref(queue), 1);
    std::thread cons2(consumer, std::ref(queue), 2);

    prod1.join();
    prod2.join();
    cons1.join();
    cons2.join();

    return 0;
}
```

```
['Producer 1 pushed: 100\nProducer 2 pushed: 200\nConsumer 1 popped: 100\nProducer 1 pushed: 101\nConsumer 2 popped: 20
```

topic_02_thread_safe_bounded_queue

Code Examples

📌 2. Timeout-Based Processing (1/2)

This example demonstrates **timed waiting with timeouts** using `pop_for()`, which is essential for building robust systems that need to detect and handle periods of inactivity. This pattern prevents threads from blocking indefinitely and allows graceful handling of scenarios where producers stop sending data.

What this code does:
- Consumer thread calls `pop_for(2s)` - waits up to 2 seconds for an item
- If an item arrives within 2 seconds, process it and continue
- If 2 seconds pass with no item, `pop_for()` returns `std::nullopt` and consumer exits
- Producer sends 3 items spaced 500ms apart, then stops
- After processing the 3rd item, consumer waits 2 seconds, times out, and exits gracefully

Key concepts demonstrated:
- **Timeout handling**: Using `std::optional` to distinguish "got value" vs "timeout"
- **Graceful termination**: Consumer can detect when producer has stopped without explicit signaling
- **Resource management**: Thread automatically exits after detecting inactivity
- **Non-blocking with timeout**: Better than infinite blocking (responsive) or busy-wait (CPU-efficient)

Why 2-second timeout:
- Items arrive every 500ms (3 items = 1.5 seconds)
- After last item, no more data for 2+ seconds → consumer detects end of stream
- Timeout duration trades off responsiveness vs tolerance for bursty data

Real-world applications:
- **Network server**: Close idle connections after N seconds of no data
- **Batch processor**: Commit accumulated data if no new items arrive within timeout
- **Health monitoring**: Alert if expected heartbeat messages stop arriving
- **Resource cleanup**: Detect stalled producers and release resources

vs. `close()` method:
- **Timeout approach**: Consumer decides when to stop (autonomous)
- **`close()` approach**: Producer explicitly signals shutdown (coordinated)
- Use timeouts when producers may fail unexpectedly or when autonomous shutdown is desired

Output:

---

topic_02_thread_safe_bounded_queue

Code Examples

📌 2. Timeout-Based Processing (2/2)

```
#include <iostream>
#include <thread>

void timed_consumer(BoundedQueue<std::string>& queue) {
    using namespace std::chrono_literals;

    while (true) {
        auto item = queue.pop_for(2s);  // Wait up to 2 seconds

        if (item) {
            std::cout << "Processed: " << *item << '\n';
        } else {
            std::cout << "Timeout - no items for 2 seconds\n";
            break;  // Exit after timeout
        }
    }
}

int main() {
    BoundedQueue<std::string> queue(10);

    std::thread consumer(timed_consumer, std::ref(queue));

    // Producer sends a few items then stops
    queue.push("Task 1");
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    queue.push("Task 2");
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    queue.push("Task 3");

    // Consumer will timeout after 2s of no items
    consumer.join();

    return 0;
}
```

```
['Processed: Task 1\nProcessed: Task 2\nProcessed: Task 3\nTimeout - no items for 2 seconds']
```

topic_02_thread_safe_bounded_queue

Code Examples

📌 3. Graceful Shutdown (1/3)

This example demonstrates **coordinated shutdown** using the `close()` method to signal all waiting threads to exit gracefully. This is the preferred approach for cleanly terminating a multi-threaded application with active worker threads, avoiding resource leaks and ensuring all threads join properly.

What this code does:
- Spawns 3 worker threads that continuously call `pop()` to fetch and process tasks
- Main thread pushes 10 tasks to the queue
- After 2 seconds, main thread calls `queue.close()` to initiate shutdown
- `close()` sets `is_closed_` flag and wakes ALL waiting threads with `notify_all()`
- Workers' `pop()` calls throw `std::runtime_error("Queue is closed and empty")`
- Workers catch the exception, print exit message, and return (thread terminates)
- Main thread joins all worker threads and confirms clean shutdown

Key shutdown patterns demonstrated:
- **Broadcast signal**: `close()` uses `notify_all()` to wake all threads simultaneously
- **Exception-based termination**: Workers exit via exception rather than return value checking
- **Resource cleanup**: All threads properly join before program exits
- **No busy-wait**: Workers sleep in `pop()` until woken by `close()`

What happens during `close()`:
1. Acquire mutex (thread-safe state change)
2. Set `is_closed_ = true`
3. Call `not_full_.notify_all()` → wake all waiting producers
4. Call `not_empty_.notify_all()` → wake all waiting consumers
5. Release mutex
6. All waiting `wait()` calls wake up, re-check predicates, see `is_closed_`, throw exception

Why exception-based termination:
- **Propagates immediately**: Unlike return value, exception unwinds stack regardless of code structure
- **Hard to ignore**: Forces caller to handle shutdown (can't accidentally continue)
- **RAII-friendly**: Exception unwinding triggers destructors, ensuring cleanup
- **Clear intent**: Exception type communicates "abnormal termination" vs "normal empty queue"

Real-world applications:
- **Thread pool shutdown**: Signal all workers to stop processing tasks
- **Server termination**: Close request queue on SIGINT/SIGTERM, drain existing work
- **Pipeline stages**: Coordinated shutdown of multi-stage processing pipelines
- **Test cleanup**: Reliably terminate test threads without hanging

Common pitfall: Forgetting to call `close()` before joining threads leads to **deadlock** - workers block forever in `pop()` waiting for items that

topic_02_thread_safe_bounded_queue

Code Examples

📌 3. Graceful Shutdown (2/3)

```
#include <iostream>
#include <thread>
#include <vector>

void worker(BoundedQueue<int>& queue, int id) {
    try {
        while (true) {
            int task = queue.pop();
            std::cout << "Worker " << id << " processing: " << task << '\n';
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
    } catch (const std::runtime_error& e) {
        std::cout << "Worker " << id << " exiting: " << e.what() << '\n';
    }
}

int main() {
    BoundedQueue<int> queue(5);

    std::vector<std::thread> workers;
    for (int i = 0; i < 3; ++i) {
        workers.emplace_back(worker, std::ref(queue), i);
    }

    // Add some tasks
    for (int i = 0; i < 10; ++i) {
        queue.push(i);
    }

    std::this_thread::sleep_for(std::chrono::seconds(2));

    // Shutdown: close queue to wake all waiting threads
    std::cout << "Shutting down...\n";
    queue.close();

    for (auto& t : workers) {
        t.join();
    }

    std::cout << "All workers stopped\n";
    return 0;
}
```

topic_02_thread_safe_bounded_queue

Code Examples

📌 3. Graceful Shutdown (3/3)

```
['Worker 0 processing: 0\nWorker 1 processing: 1\nWorker 2 processing: 2\n...\nShutting down...\nWorker 0 exiting: Queu
```

What You Learned

Key Takeaways

- ✓ The Producer-Consumer Problem
- ✓ Synchronization Primitives
- ✓ **std::mutex**
- ✓ **std::condition_variable**

→ Swipe to learn →